



The Reader Monad — Example & Motivation



Kyle Corbelli · Follow

Published in DSC Engineering

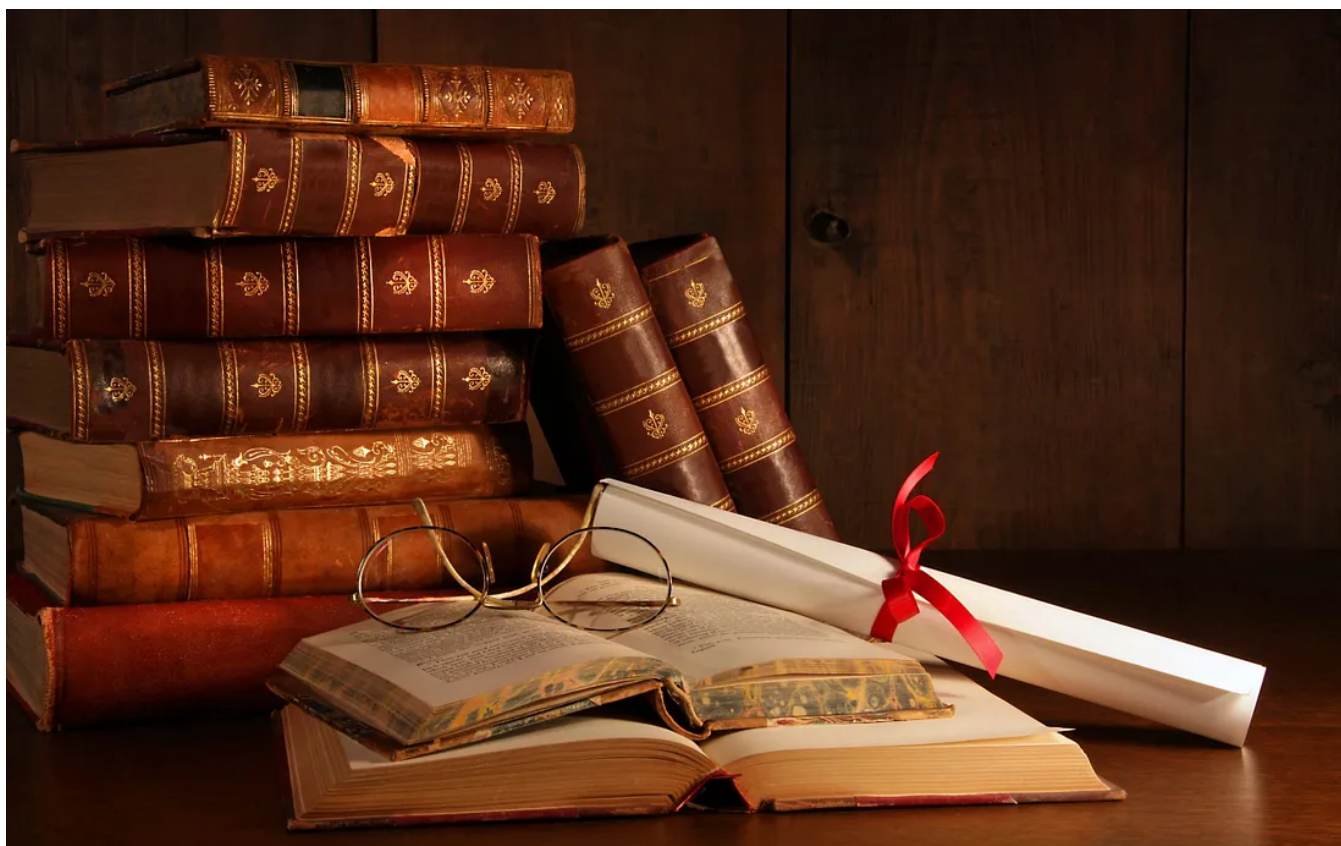
7 min read · Jul 2, 2018



Listen



Share



Intro

Can't find a solid example of the Reader monad in action? Yeah, welcome to the club. Here, we'll put together a simple example showcasing the utility and delight of the Reader monad.

Table of Contents

Just Give Me the Code

1. What is the Reader Monad Good For?

2. Example

- Setting the Scene
- First Pass
- Reader Monad Implementation Details (optional)
- Refactor Using the Reader Monad
- Refactor Using “do” Notation

3. Conclusion

What is the Reader Monad Good For?

Yo dawg, I heard you like functions... so I wrote a program for you using functions that call functions that call other functions that then call more functions. And guess what? I passed in some value as a parameter to the first function that is then passed in as a parameter to the functions *that* function calls. So we find ourselves passing around some “shared” value from function to function. But what if one of the functions doesn’t need that shared value for any reason other than to pass it to some other function that *does* need it? For example, in the following code snippet `first` and `third` don’t use `name` for anything other than passing it to other functions. `second` and `fourth`, however, *do* use `name` directly:

```
1  main :: IO ()
2  main = putStrLn $ first "Buddy"
3
4  type Name = String
5
6  first :: Name -> String
7  first name =
8      second name ++ third name
9
10 second :: Name -> String
11 second name =
12     "Welcome" ++ name ++ "!"
13
14 third :: Name -> String
15 third name =
16     (fourth name) ++ "!"
17
18 fourth :: Name -> String
19 fourth name =
20     " I wish my name was " ++ name
```

shared-environmental-state.hs hosted with ♥ by GitHub

[view raw](#)

In such a small example this isn't particularly problematic, but as the code grows larger and larger, with longer function invocation chains, taking in and passing along some shared value can be cumbersome.

The Reader monad is useful when you have a long chain of function invocations where only some of the functions in the chain make direct use of a piece of “shared environmental state”. This “environmental state” could be anything. It could be as simple as a primitive value or it could be a record containing configuration values your program needs in certain places. Doesn't matter. Regardless of what that environmental state is, it needs to be passed through to each function along the way so that each function has access to it. Now, whether that particular function actually makes *direct use* of the environmental state or simply needs to receive it in order to pass it along to another function is up to each individual function.

If you've written a web application in React or any number of component-based

frontend libraries, you're probably familiar with passing `props` down through components. Have you ever passed a `prop` from a parent component to a child component *just because a grandchild component needs it*? The middle component couldn't care less about the `prop` but you end up passing it through anyway. And this might go several layers deep! Kind of a chore, right? If nothing else, it's boilerplate. And that's exactly what the Reader monad aims to reduce.

Example

Setting the Scene

We'll illustrate the utility of the Reader monad by contriving an example that leads to a simple `String` being passed through a bunch of functions, *only some of which actually make direct use of it*. Our little program's domain will be creating dynamic markup for the web. We'll prompt the user for their email address on the command line and then we'll generate some HTML using a homemade API somewhat similar to React's JSX API. We'll start by first creating a teeny-tiny, bare-bones HTML templating library in Haskell.

```
1  module Html where
2
3  import      Control.Monad.Reader (Reader, ask, runReader)
4  import      Data.List    (intercalate)
5  import      Prelude      hiding (div)
6
7  type Html = String
8  type Email = String
9
10 div :: [Html] -> Html
11 div children =
12     "<div>" ++ combine children ++ "</div>"
13
14 h1 :: [Html] -> Html
15 h1 children =
16     "<h1>" ++ combine children ++ "</h1>"
17
18 p :: [Html] -> Html
19 p children =
20     "<p>" ++ combine children ++ "</p>"
21
22 combine :: [Html] -> Html
23 combine = intercalate ""
```

haskell-simple-html.hs hosted with ♥ by [GitHub](#)

[view raw](#)

All we're doing here is type aliasing `Html` and `Email` as `String`, then creating three functions that act like stripped down HTML elements `div`, `h1`, and `p`. Our little functions don't allow attributes or anything fancy at all. They just act like HTML tags, which accept a list of other HTML tags. If you've written applications in Elm or PureScript (and I can't recommend this highly enough) then this should be quite familiar to you. Now we can use this lil' API to create some simple web content with the following hierarchy. Keep in mind that we'll be passing the user's `Email` (remember, just a `String` alias) to the very top level `view` function where it will proceed to pass it down the hierarchy:

- view
 - page
 - topnav
 - content (*needs email*)
 - left
 - right
 - article
 - widget (*needs email*)

`haskell-simple-webpage-hierarchy.md` hosted with ♥ by [GitHub](#)

[view raw](#)

First Pass

We'll create a named view function for each piece of the hierarchy. If you're from the React world, think of each view function as a stateless functional component. As you check out the following code, take note that since the `widget` view function all the way down at the bottom needs the user's `Email`, we'll need to pass it through all of its "ancestor" functions along the way:

```
1  main :: IO ()
2  main =
3      putStrLn "what is your email address?" >>
4          getLine >=> \email ->
5              putStrLn . show $ view email
6
7  view :: Email -> Html
8  view email =
9      div
10         [ page email
11         ]
12
13  page :: Email -> Html
14  page email =
15      div
16         [ topNav
17           , content email
18         ]
19
20  topNav :: Html
21  topNav =
22      div
23         [ h1 [ "OurSite.com" ]
24         ]
25
26  content :: Email -> Html
27  content email =
28      div
29         [ h1 [ "Custom Content for " ++ email ]
30           , left
31           , right email
32         ]
33
34  left :: Html
35  left =
36      div
37         [ p [ "this is the left side" ]
38         ]
39
40  right :: Email -> Html
41  right email =
42      div
```

```
43     [ article email
44     ]
45
46 article :: Email -> Html
47 article email =
48     div
49     [ p [ "this is an article" ]
50       , widget email
51     ]
52
53 widget :: Email -> Html
54 widget email =
55     div
56     [ p [ "Hey " ++ email ++ ", we've got a great offer for you!" ]
57     ]
```

`haskell-simple-webpage.hs` hosted with ❤ by [GitHub](#)

[view raw](#)

The only view functions that actually need the `Email` directly are `content` and `widget` ! All the others just pass it along faithfully without using it. See `article` for a specific example. It doesn't make use of `email` other than to pass it to `widget` . Wouldn't it be nice to avoid all the take-and-pass boilerplate? Enter the Reader monad.

Reader Monad Implementation Details (optional)

Now, I don't really want to get hung up on the implementation of the Reader monad itself because that may confuse the example. Nevertheless, for the sake of completeness, here's a [sample Reader monad implementation](#). The actual Haskell implementation is more involved but perhaps less instructive.

Refactor Using the Reader Monad

If we take the Reader monad as granted we can update the type signatures of many of our view functions from `Email -> Html` to `Reader Email Html` . This is to say, we have a Reader whose “shared environmental state” is of type `Email` and when “run” with such a value, it will return a value of type `Html` . It's pretty much a direct analogue of `Email -> Html` , which says “I take in a parameter of type `Email` and return a value of type `Html` ”. Tomato/Tomáto.

Now that none of our view functions explicitly take in an `Email` , we'll remove `email`

as a parameter in the function signatures. We'll also update the implementation details of each view function to make use of the fact that it is actually now an official Reader monad. Finally, we need to call `runReader` on our top-level `view Reader` in `main` in order to “extract” the function wrapped inside of it. Then we can pass in `email` to that function:

```
1  main :: IO ()
2  main =
3      putStrLn "what is your email address?" >>
4          getLine >>= \email ->
5              putStrLn . show $ runReader view email
6
7  view :: Reader Email Html
8  view =
9      page >>= \page' ->
10         return $ div
11             [ page'
12             ]
13
14  page :: Reader Email Html
15  page =
16      content >>= \content' ->
17         return $ div
18             [ topNav
19               , content'
20             ]
21
22  topNav :: Html
23  topNav =
24      div
25          [ h1 [ "OurSite.com" ]
26          ]
27
28  content :: Reader Email Html
29  content =
30      ask >>= \email ->
31          right >>= \right' ->
32              return $ div
33                  [ h1 [ "Custom Content for " ++ email ]
34                    , left
35                    , right'
36                  ]
37
38  left :: Html
39  left =
40      div
41          [ p [ "this is the left side" ]
42          ]
```

```

43
44 right :: Reader Email Html
45 right =
46   article >>= \article' ->
47     return $ div
48       [ article'
49       ]
50
51 article :: Reader Email Html
52 article =
53   widget >>= \widget' ->
54     return $ div
55       [ p [ "this is an article" ]
56       , widget'
57       ]
58
59 widget :: Reader Email Html
60 widget =
61   ask >>= \email ->
62     return $ div
63       [ p [ "Hey " ++ email ++ ", we've got a great offer for you!" ]
64       ]

```

`haskell-simple-webpage-reader-monad` is hosted with ❤ by [GitHub](#)

[view raw](#)

Whoa! What on earth is happening here? It might be easier to start from the bottom and work our way up. Let's start with the `widget` Reader. Since it *does* actually need the `Email`, we can access it using the `ask` Reader (typically defined in the `Control.Monad.Reader` module), feeding it into the `>>=` function as defined for the Reader monad. When run, the `ask` Reader applies the identity function `id`, so after being passed through `>>=`, we'll end up getting the `Email` value itself bound to the parameter in the lambda. From there we can use it like any other `String`.

Moving one layer up we inspect the `article` view function. This function does *not* need the `Email` value itself so we don't `ask` for it. It plain never shows up. It does, however, need the markup provided by `widget`. Since `widget` is no longer a function and is not of type `Html` but rather of type `Reader Email Html`, we need a way to access the returned `Html` since `div` expects a list of `Html` values. Luckily, this is exactly what `>>=` as defined for the Reader monad does! So we feed `widget` into it, and the value bound to the parameter in the lambda (`widget'`) is the eventual `Html`

returned from `widget`. Now within the lambda we can treat it like a good ol' `Html` value and include it in the list of `Html` values we're giving `div` here.

This same tactic is employed another level up in the `right` Reader, which doesn't need the `Email` itself either. We just feed `article` to `>>=` and bind the returned `Html` to the parameter in the lambda (`article'`) for use however we see fit.

Perhaps the most interesting Reader here is `content`, which needs both the `Email` itself *and* the rendered `Html` from descendent Readers. To get the `Email` value, just ask for it, passing `ask` through `>>=`. Then within *that* lambda grab the `right` Reader and pass *it* through `>>=`. Now you have access to both bound values via the closure scope. Proceed as usual.

Looking at our code now, we see that all of the intermediate Readers that don't make direct use of the `Email` value don't even mention it. They just go about their business ignoring it but allowing descendant Readers to access it should they so desire. Boilerplate sniped.

Refactor Using “do” Notation

This can, of course, be cleaned up even further using `do` notation:

```
1  main :: IO ()
2  main = do
3    putStrLn "what is your email address?"
4    email <- getLine
5    putStrLn . show $ runReader view email
6
7  view :: Reader Email Html
8  view = do
9    page' <- page
10   return $ div
11     [ page'
12     ]
13
14  page :: Reader Email Html
15  page = do
16    content' <- content
17    return $ div
18      [ topNav
19      , content'
20      ]
21
22  topNav :: Html
23  topNav =
24    div
25      [ h1 [ "OurSite.com" ]
26      ]
27
28  content :: Reader Email Html
29  content = do
30    email <- ask
31    right' <- right
32    return $ div
33      [ h1 [ "Custom Content for " ++ email ]
34      , left
35      , right'
36      ]
37
38  left :: Html
39  left =
40    div
41      [ p [ "this is the left side" ]
42      ]
```

```
43
44 right :: Reader Email Html
45 right = do
46   article' <- article
47   return $ div
48     [ article'
49     ]
50
51 article :: Reader Email Html
52 article = do
53   widget' <- widget
54   return $ div
55     [ p [ "this is an article" ]
56       , widget'
57     ]
58
59 widget :: Reader Email Html
60 widget = do
61   email <- ask
62   return $ div
63     [ p [ "Hey " ++ email ++ ", we've got a great offer for you!" ]
64     ]
```

`haskell-simple-webpage-reader-monad-do-notation` is hosted with ❤ by [GitHub](#)

[view raw](#)

Instead of using the (potentially nested) lambdas following invocations of `>>=`, we unpack the `Email` value and descendant `Reader Html` values with `<-`. I won't go into the specifics of `do` notation here but it is arguably cleaner. Just know that it's nothing more than syntactic sugar for `>>=` with lambdas.

Conclusion

And there it is. The Reader monad lets us quietly pass down “shared environmental state” through a bunch of functions, allowing each one along the way to decide whether or not it cares about it specifically or just needs to silently keep passing it along. [Check out the full code example, which adds the ability to write our dynamically created markup to an HTML file.](#)

What do you think?

Haskell

Monads

Functional Programming

React

Elm